
Convolutional Neural Networks

— Dr. Anuradha Yenikar —

Introduction

Image classification is the task of taking an input image and **outputting a class** or a **probability of classes** that best describes the image.

For humans, this task is one of the first skills we learn and it comes **naturally and effortlessly** as adults.

Being able to quickly **recognize patterns**, generalize from **prior knowledge**, and adapt to **different image environments** are difficult tasks for machines

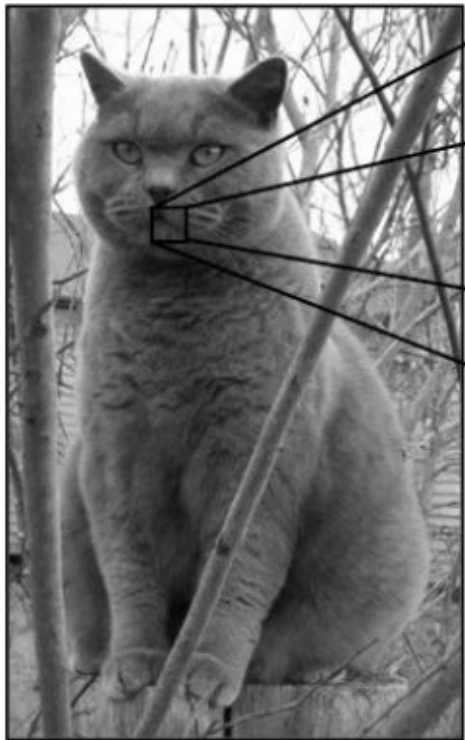


Introduction

What we see

vs.

What computers see



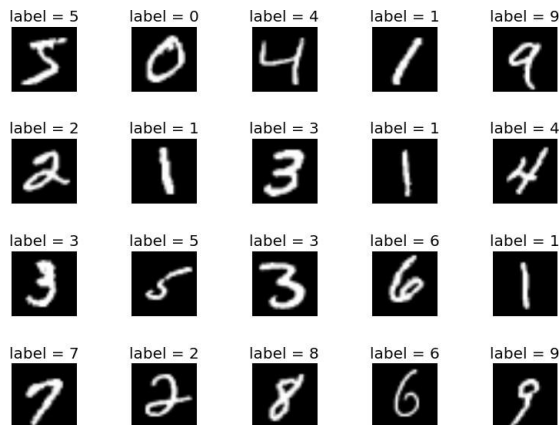
05	02	22	97	38	15	00	40	00	75	04	05	07	78	52	12	50	77	51	74
49	49	99	40	17	81	18	57	60	87	17	40	98	43	69	43	54	56	62	00
81	49	31	73	55	79	14	29	93	71	40	67	52	28	30	03	49	13	36	65
92	70	95	23	04	60	11	52	69	21	85	56	01	32	56	71	37	02	36	91
22	31	16	71	51	67	03	89	41	92	36	54	22	40	40	28	66	33	13	80
24	47	33	80	99	03	45	02	44	75	33	53	78	36	84	20	35	17	12	50
32	98	81	28	44	23	67	10	26	38	40	67	59	54	70	66	18	38	64	70
67	26	20	68	02	62	12	20	95	63	94	39	63	08	40	91	66	49	94	21
24	55	58	05	66	73	99	26	97	17	78	78	96	83	14	88	34	89	63	72
21	36	23	09	75	00	76	44	20	45	35	14	00	61	33	97	34	31	33	95
78	17	53	28	22	75	31	67	15	94	03	80	04	62	16	14	09	53	56	92
16	39	05	42	96	35	31	47	55	58	88	24	00	17	54	24	36	29	85	57
86	56	00	48	35	71	89	07	05	44	44	37	44	60	21	58	51	54	17	58
19	80	81	68	05	94	47	69	28	73	92	13	86	52	17	77	04	89	55	40
04	52	08	83	97	35	99	16	07	97	57	32	16	26	26	79	33	27	98	66
55	34	65	87	57	62	20	72	03	46	33	67	46	55	12	32	63	93	53	49
04	42	16	73	25	45	39	11	24	94	72	18	08	46	29	32	40	62	76	36
20	69	36	41	72	30	23	88	34	45	83	69	82	67	59	85	74	04	36	16
20	73	35	29	78	31	90	01	74	31	49	71	45	54	51	16	23	57	05	54
01	70	54	71	83	51	54	69	16	92	33	48	61	43	52	01	89	19	67	48

What the computer sees

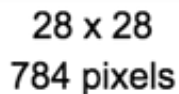
Introduction

MNIST Dataset (<http://yann.lecun.com/exdb/mnist/>)

- 60,000 training examples
- 10,000 test examples
- Rank of best Classifiers and Errors
- Currently Best Accuracy:
 - Ciresan et al. CVPR 2012 □ **99.77%**



MNIST Dataset

[illegible]

Introduction

CIFAR-10 Dataset (<https://www.cs.toronto.edu/~kriz/cifar.html>)

- Consists of 60,000 32x32 **color** images in 10 classes, with 6,000 images per class. There are 50,000 training images and 10,000 test images.



Introduction

Color Images

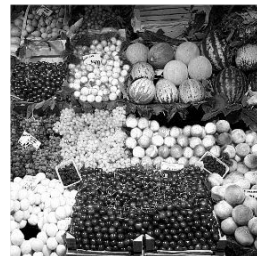
RGB



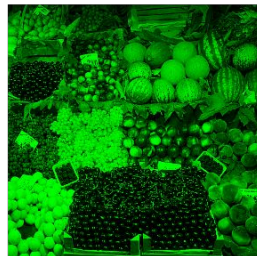
R



Red



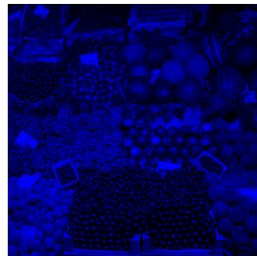
G



Green



B

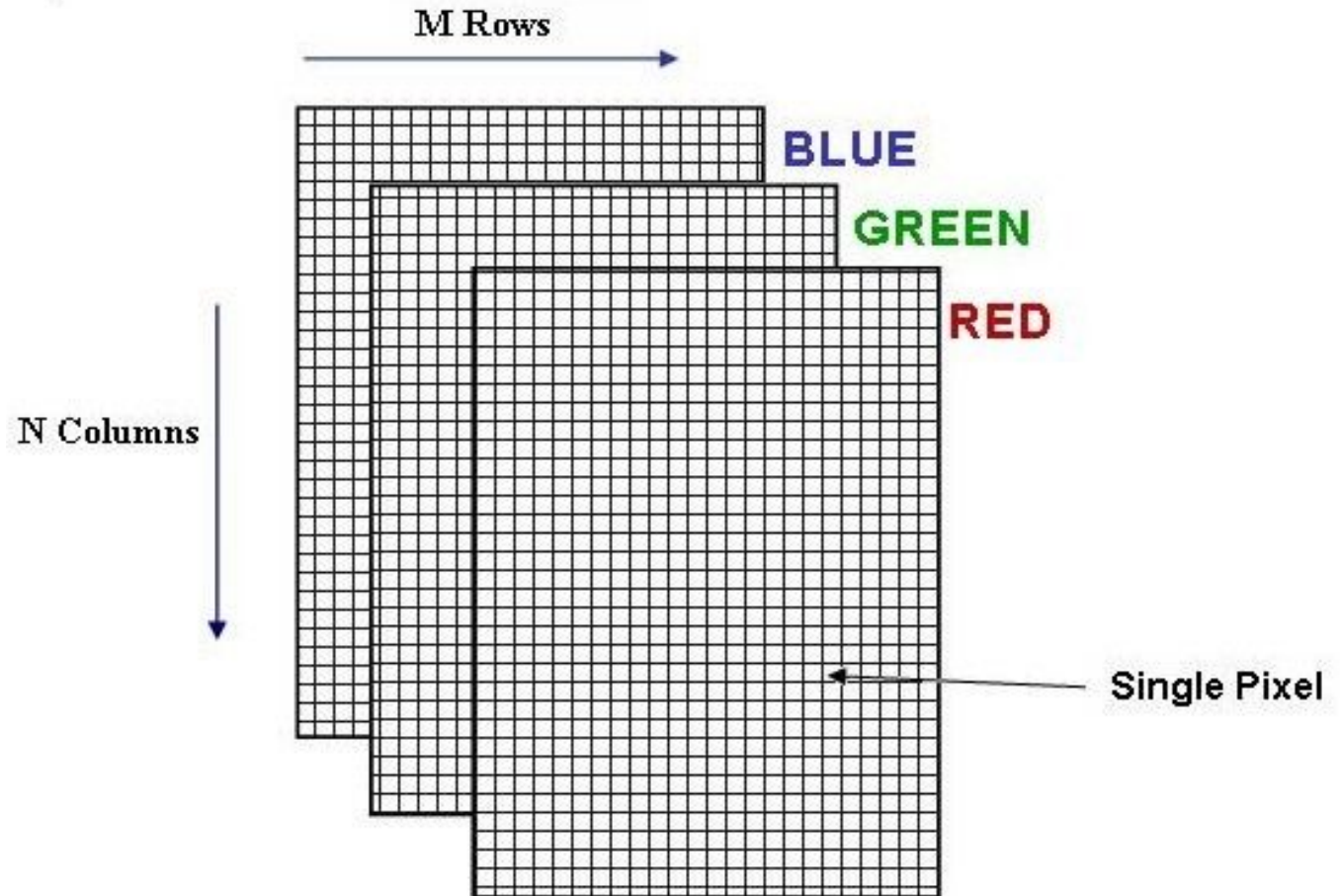


Blue



Introduction

Color Images



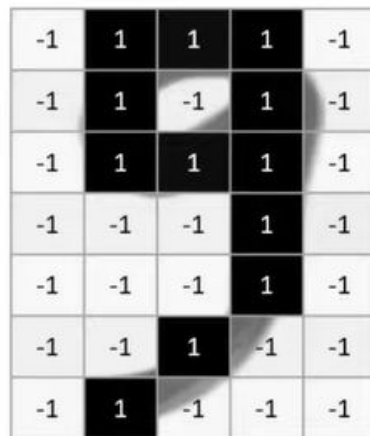
Task - Teach Computer / Machine to Identify Number 9



Solution? - Deep learning - ANN

Image credit: <https://mathematicaforprediction.files.wordpress.com/2013/08/digitimageswithzenbrush-testset.jpg>

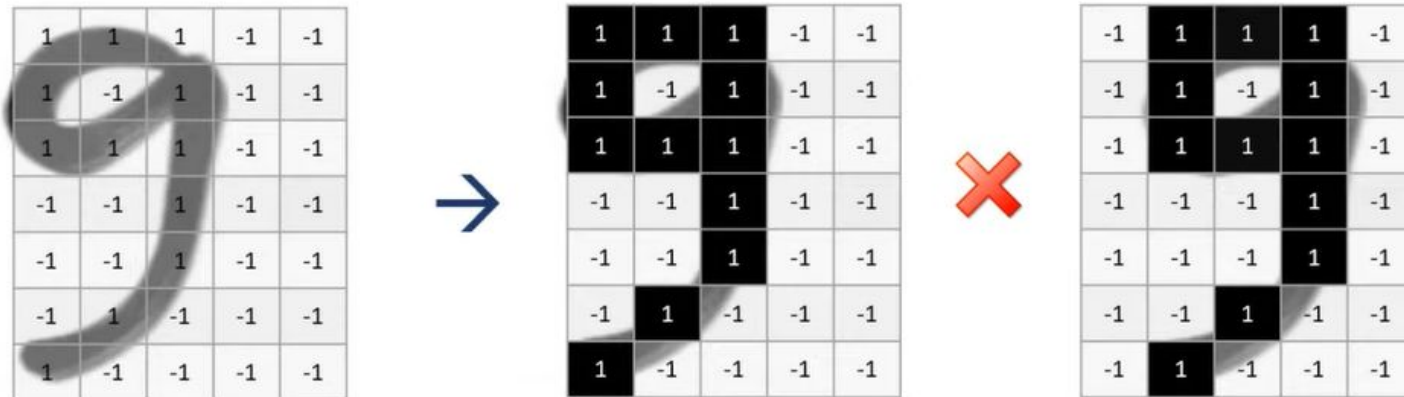
Computer creates image pixel map and feature map



Feature Map

But if ...

Location shifted



For ANN - difficult to handle when location is shifted

Variation 1

-1	-1	1	-1	-1
-1	1	-1	1	-1
-1	1	1	-1	-1
-1	-1	1	-1	-1
-1	-1	1	-1	-1
-1	-1	1	-1	-1
-1	-1	1	-1	-1



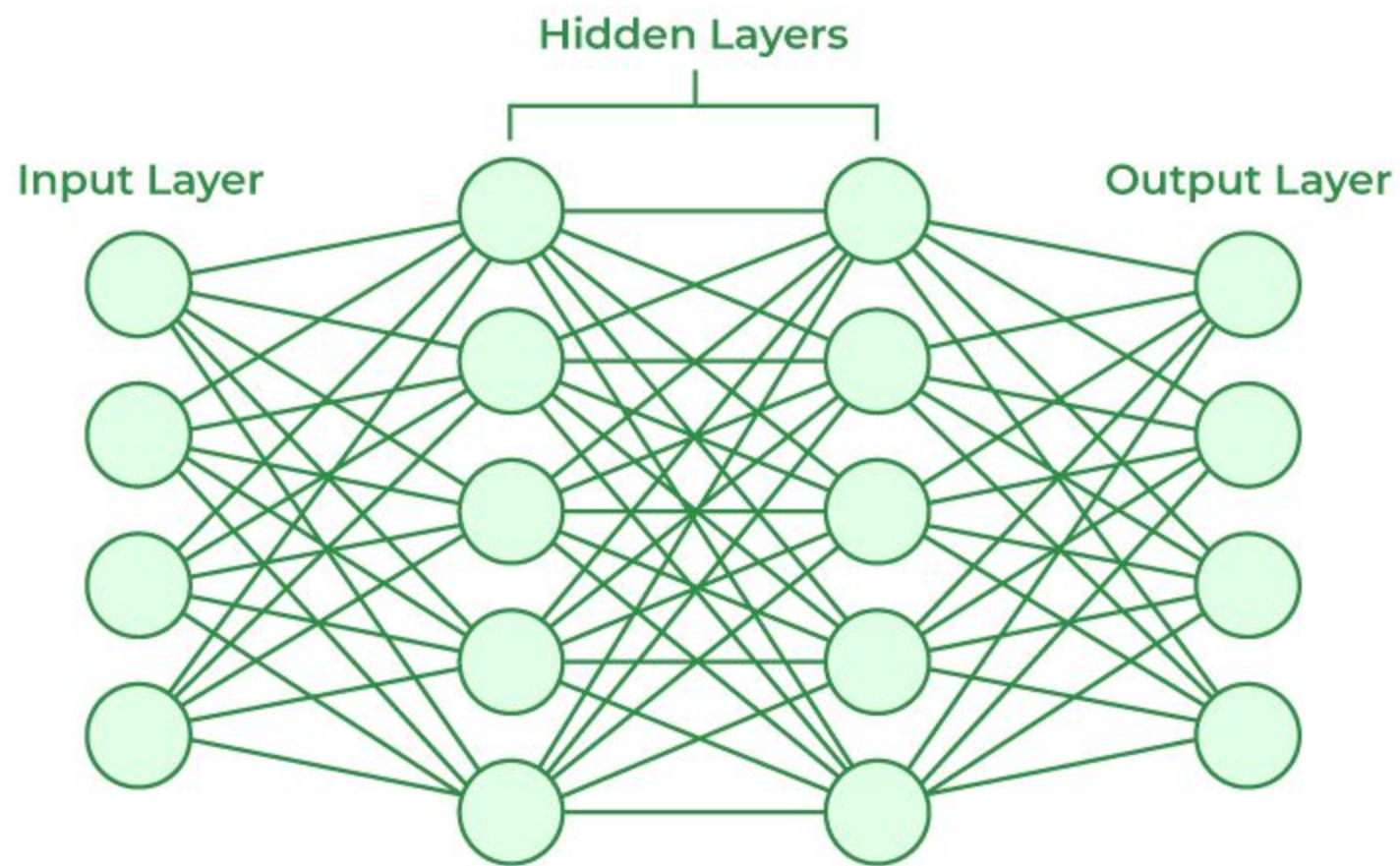
-1	-1	1	-1	-1
-1	1	-1	1	-1
-1	1	1	-1	-1
-1	-1	1	-1	-1
-1	-1	1	-1	-1
-1	-1	1	-1	-1
-1	-1	1	-1	-1



-1	1	1	1	-1
-1	1	-1	1	-1
-1	1	1	1	-1
-1	-1	-1	1	-1
-1	-1	-1	1	-1
-1	-1	1	-1	-1
-1	1	-1	-1	-1



For ANN - difficult to handle when there is variation



Identify '2'



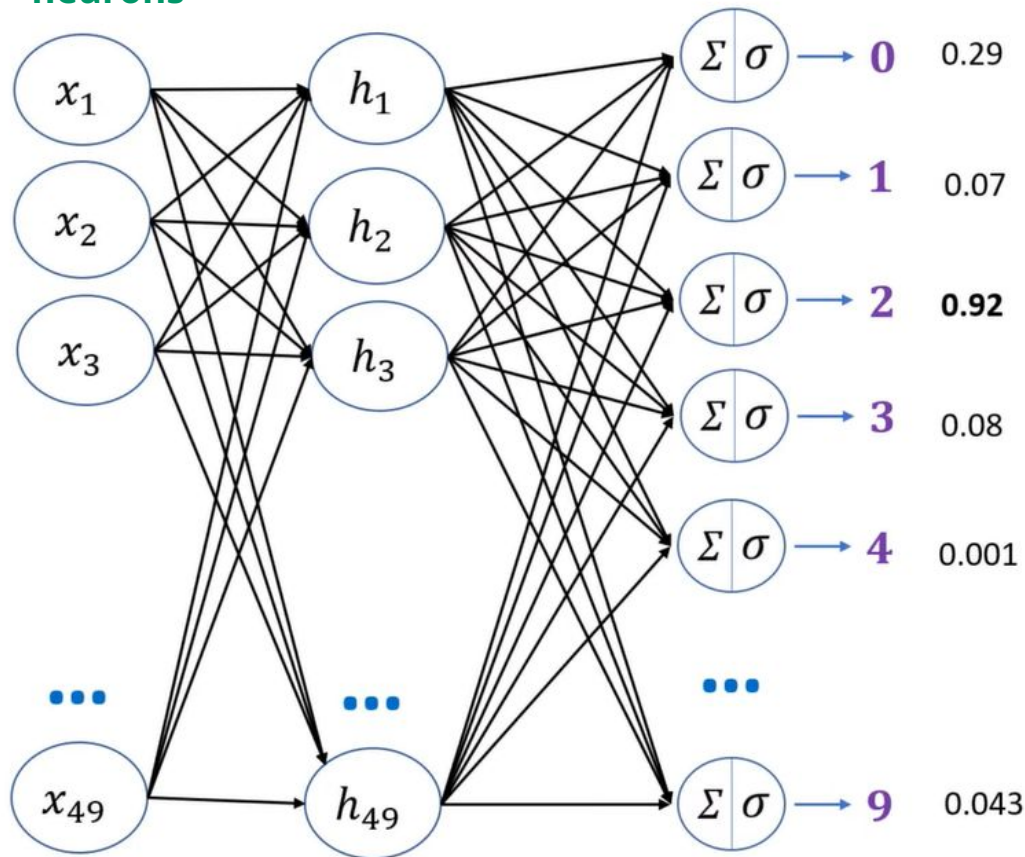
0	0	0	0	0	0	0
0	87	240	210	24	0	0
0	13	0	101	195	0	0
0	35	167	99	210	0	0
0	145	230	240	201	189	140
0	0	102	67	17	13	0
0	0	0	0	0	0	0

7 by 7 grid

7 x 7 Pixel Image

0
0
0
0
0
0
0
87
240
210
24
0
...
0

49 input
neurons



49*49 = 2401

49*10 = 490

This neural network becomes complex when image size is large



Image size = $1920 \times 1080 \times 3$

First layer neurons = $1920 \times 1080 \times 3 \sim 6 \text{ million}$

Hidden layer neurons = Let's say you keep it $\sim 4 \text{ million}$

Weights between input and hidden layer = $6 \text{ mil} \times 4 \text{ mil}$
 $= 24 \text{ million}$

24 Million connections in neural network.



Disadvantages of using ANN for image classification

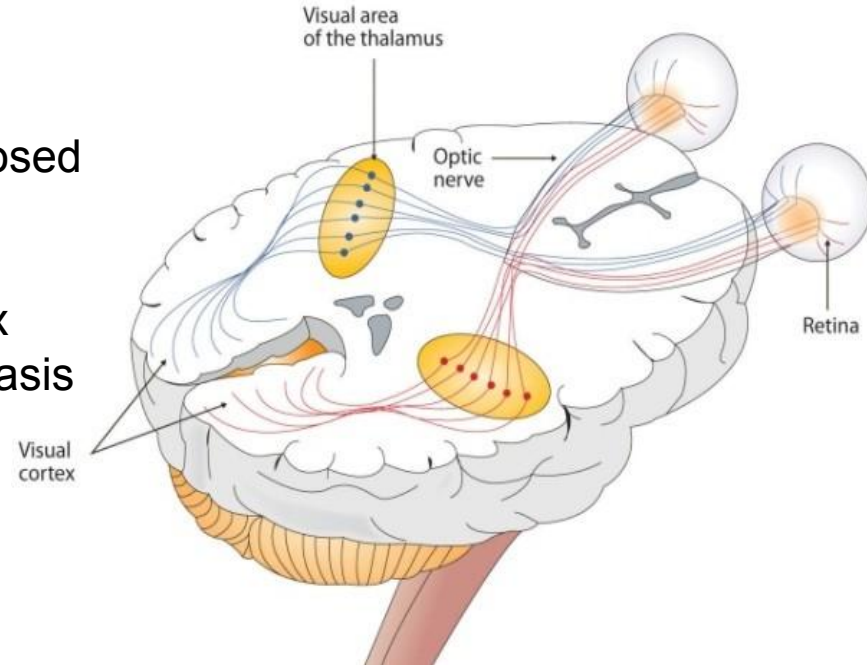
1. Too much computation
2. Treats local pixels same as pixels far apart
3. Sensitive to location of an object in an image





Convolutional Neural Networks (CNNs)

- CNNs take a **biological** inspiration from the visual cortex
- The visual cortex has small regions of cells that are sensitive to **specific regions** of the visual field
- For example, some neurons fired when exposed to **vertical** edges and some when shown **horizontal** or **diagonal** edges
- Having the neuronal cells in the visual cortex looking for **specific characteristics** is the basis behind CNNs



CNN - Divides image into Patches to learn quickly..



Koala's **eye**? = Y



Koala's **nose**? = Y



Koala's **ears**? = Y



Koala's **head**? = Y



Koala's **hands**? = Y



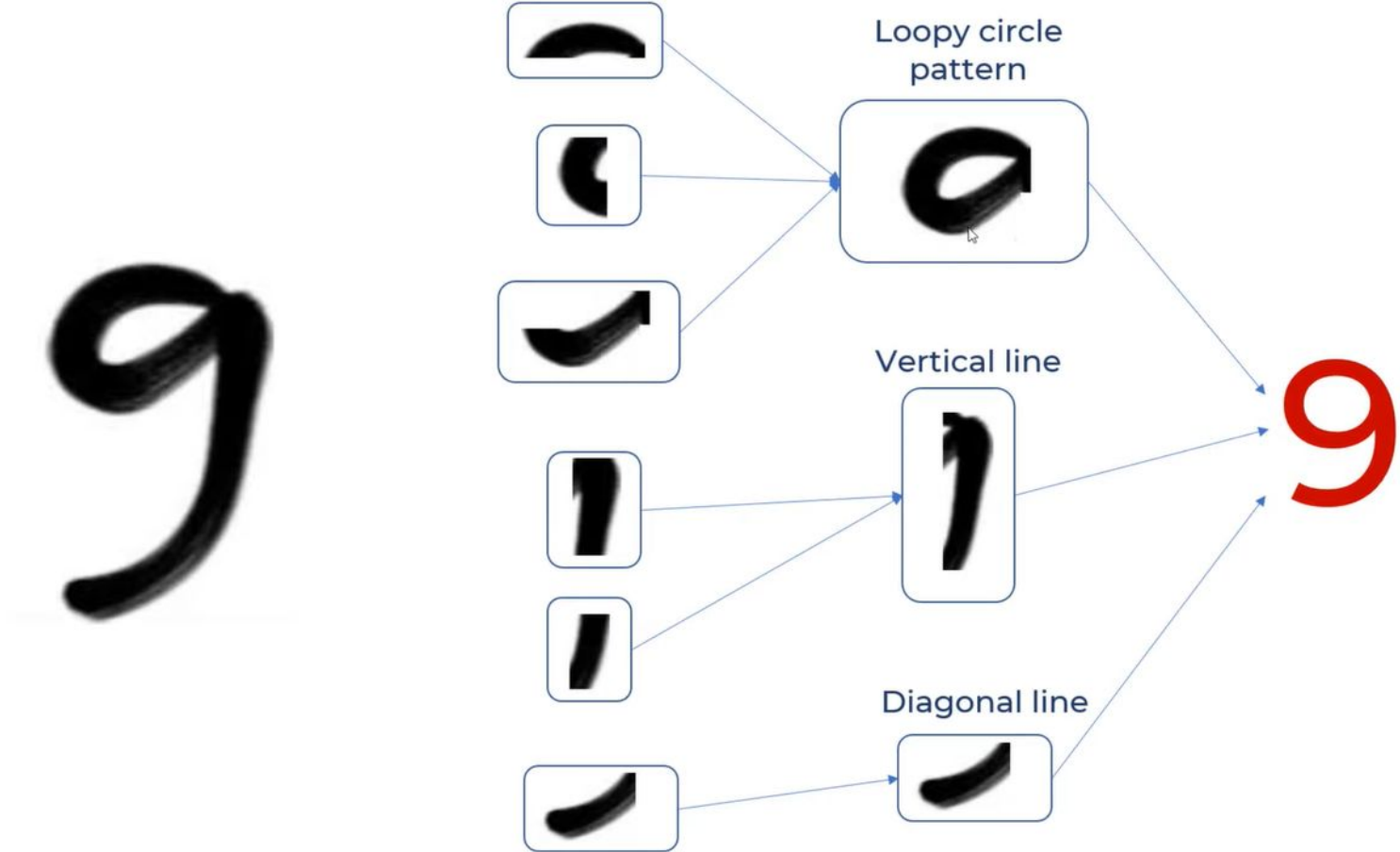
Koala's **legs**? = Y



Koala's **body**? = Y



CNN - Divides image into Patches to learn quickly..



9 - (Loopy Pattern + Vertical line + Diagonal line) Filter

-1	1	1	1	-1
-1	1	-1	1	-1
-1	1	1	1	-1
-1	-1	-1	1	-1
-1	-1	-1	1	-1
-1	-1	1	-1	-1
-1	1	-1	-1	-1

-1	1	1	1	-1
-1	1	-1	1	-1
-1	1	1	1	-1
-1	-1	-1	1	-1
-1	-1	-1	1	-1
-1	-1	1	-1	-1
-1	1	-1	-1	-1

Loopy pattern
filter

-1	1	1	1	-1
-1	1	-1	1	-1
-1	1	1	1	-1
-1	-1	-1	1	-1
-1	-1	-1	1	-1
-1	-1	1	-1	-1
-1	1	-1	-1	-1

-1	1	1	1	-1
-1	1	-1	1	-1
-1	1	1	1	-1
-1	-1	-1	1	-1
-1	-1	-1	1	-1
-1	-1	1	-1	-1
-1	1	-1	-1	-1

Vertical line
filter

-1	1	1	1	-1
-1	1	-1	1	-1
-1	1	1	1	-1
-1	-1	-1	1	-1
-1	-1	-1	1	-1
-1	-1	1	-1	-1
-1	1	-1	-1	-1

Diagonal line
filter

Image Patches -

Tiny Features of image

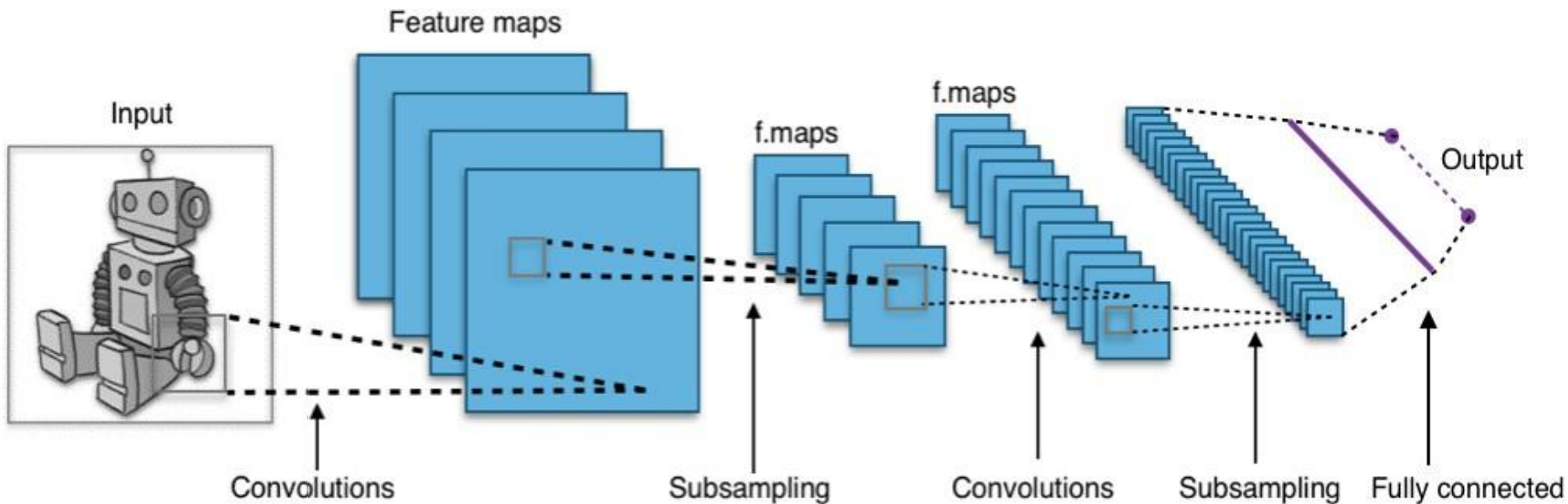
**HOW CAN WE MAKE COMPUTERS
RECOGNIZE THESE TINY FEATURES?**



Convolutional Neural Networks (CNNs)

Network Architecture

- Convolutional Layer, Pooling Layer, Fully Connected Layer



Convolutional Neural Networks (CNNs)

Convolution Operator

$$(I * K)_{xy} = \sum_{i=1}^h \sum_{j=1}^w K_{ij} \cdot I_{x+i-1, y-j-1}$$

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Input Image (I)

X

1	0	1
0	1	0
1	0	1

Filter (K)

=

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

4		

Convolved
Feature

The 3×3 matrix (K) is called a **‘filter’** or **‘kernel’** or **‘feature detector’** and the matrix formed by sliding the filter over the image and computing the dot product is called the **‘Convolved Feature’** or **‘Activation Map’** or the **‘Feature Map’**.

'Filter' or 'Kernel' or 'Feature Detector'

$$-1+1+1-1-1-1-1+1+1 = -1 \rightarrow -1/9 = -0.11$$

-1	1	1	1	-1
-1	1	-1	1	-1
-1	1	1	1	-1
-1	-1	-1	1	-1
-1	-1	-1	1	-1
-1	-1	1	-1	-1
-1	1	-1	-1	-1

*

1	1	1
1	-1	1
1	1	1

-0.11		

'Filter' or 'Kernel' or 'Feature Detector'

-1	1	1	1	-1
-1	1	-1	1	-1
-1	1	1	1	-1
-1	-1	-1	1	-1
-1	-1	-1	1	-1
-1	-1	1	-1	-1
-1	1	-1	-1	-1

*

1	1	1
1	-1	1
1	1	1

-0.11	1	

- CNN learns the values of these **filters** on its own during the **training process**
- Although we still need to specify parameters such as **number of filters**, **filter size**, **padding**, and **stride** before the training

'Filter' or 'Kernel' or 'Feature Detector'

-1	1	1	1	-1
-1	1	-1	1	-1
-1	1	1	1	-1
-1	-1	-1	1	-1
-1	-1	-1	1	-1
-1	-1	1	-1	-1
-1	1	-1	-1	-1

*

1	1	1
1	-1	1
1	1	1

-0.11	1	-0.11
-0.55	0.11	



'Filter' or 'Kernel' or 'Feature Detector'

-1	1	1	1	-1
-1	1	-1	1	-1
-1	1	1	1	-1
-1	-1	-1	1	-1
-1	-1	-1	1	-1
-1	-1	1	-1	-1
-1	1	-1	-1	-1

*

1	1	1
1	-1	1
1	1	1

-0.11	1	-0.11
-0.55	0.11	-0.33
-0.33	0.33	-0.33
-0.22	-0.11	-0.22
-0.33	-0.33	-0.33

Feature Map

Feature Map

Pattern Detector

9 * $\begin{matrix} & \text{Loopy pattern} \\ & \text{detector} \end{matrix}$

1	1	1
1	-1	1
1	1	1

=

	1	

6 * $\begin{matrix} & \text{Loopy pattern} \\ & \text{detector} \end{matrix}$

1	1	1
1	-1	1
1	1	1

=

	1	

Pattern Detector

Loopy pattern detector

9 *

1	1	1
1	-1	1
1	1	1

 =

	1	

Loopy pattern detector

6 *

1	1	1
1	-1	1
1	1	1

 =

	1	

Loopy pattern detector

8 *

1	1	1
1	-1	1
1	1	1

 =

	1	
	1	

Loopy pattern detector

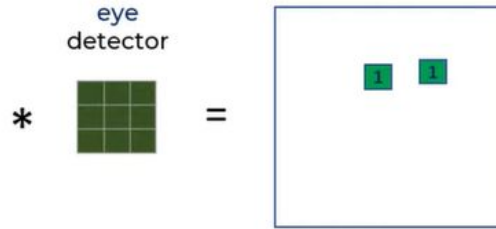
96 *

1	1	1
1	-1	1
1	1	1

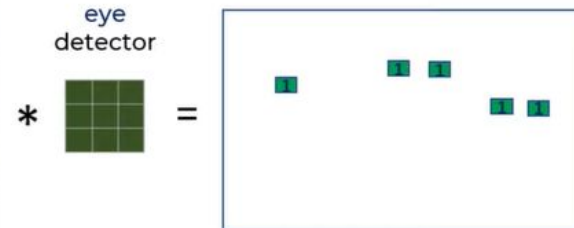
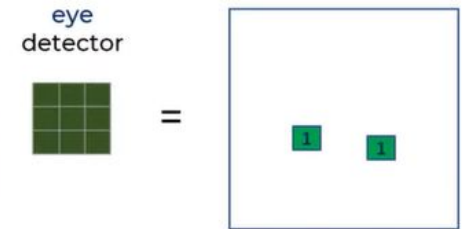
 =

	1		
		1	

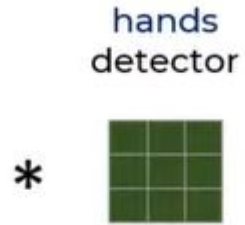
Pattern Detector



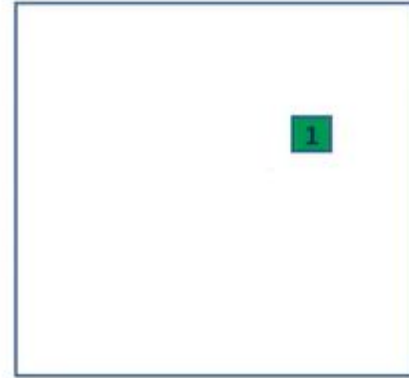
Location invariant: It can detect eyes in any location of the image



Pattern Detector



=



Pattern Detector - for all 3 patterns

Loopy pattern detector

 $*$

1	1	1
1	-1	1
1	1	1

 $=$

	1	

Vertical line detector

 $*$

-1	1	-1
-1	1	-1
-1	1	-1

 $=$

	1	

Diagonal line detector

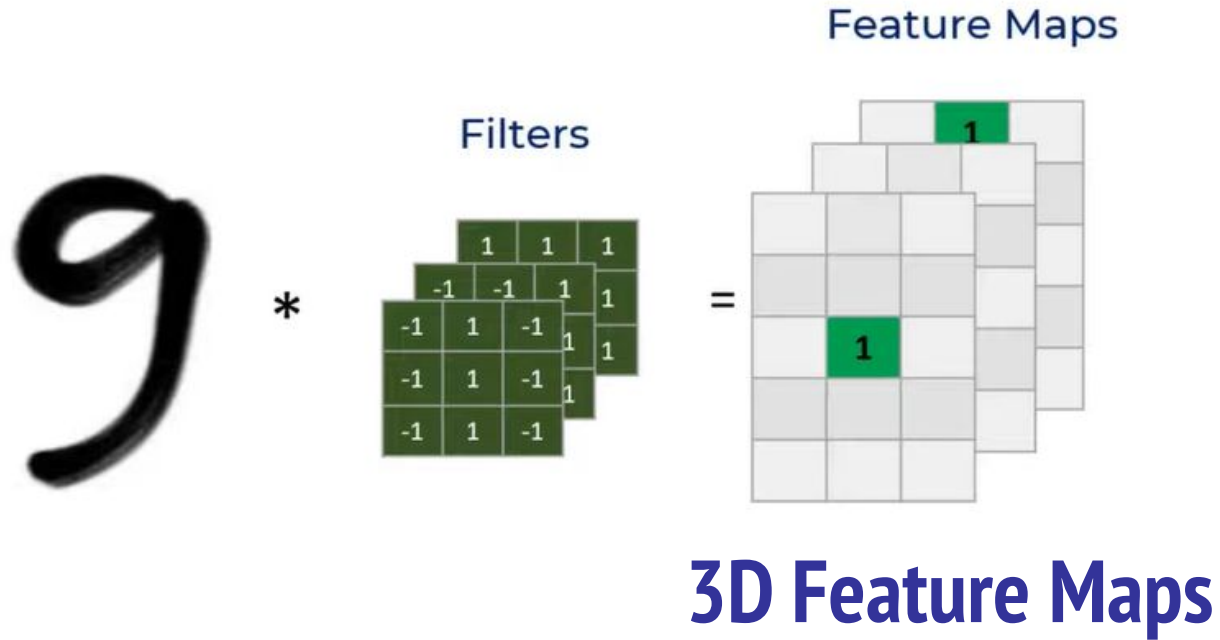
 $*$

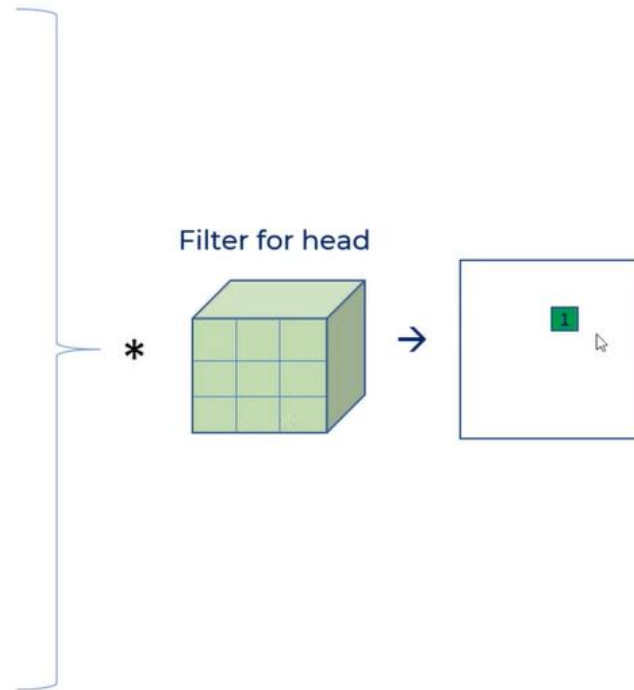
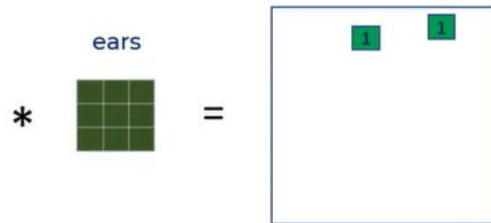
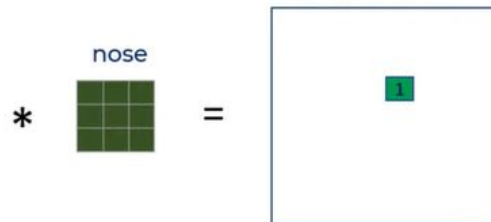
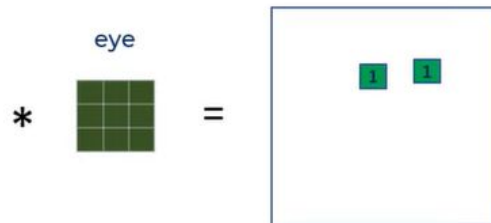
-1	-1	1
-1	1	-1
1	-1	-1

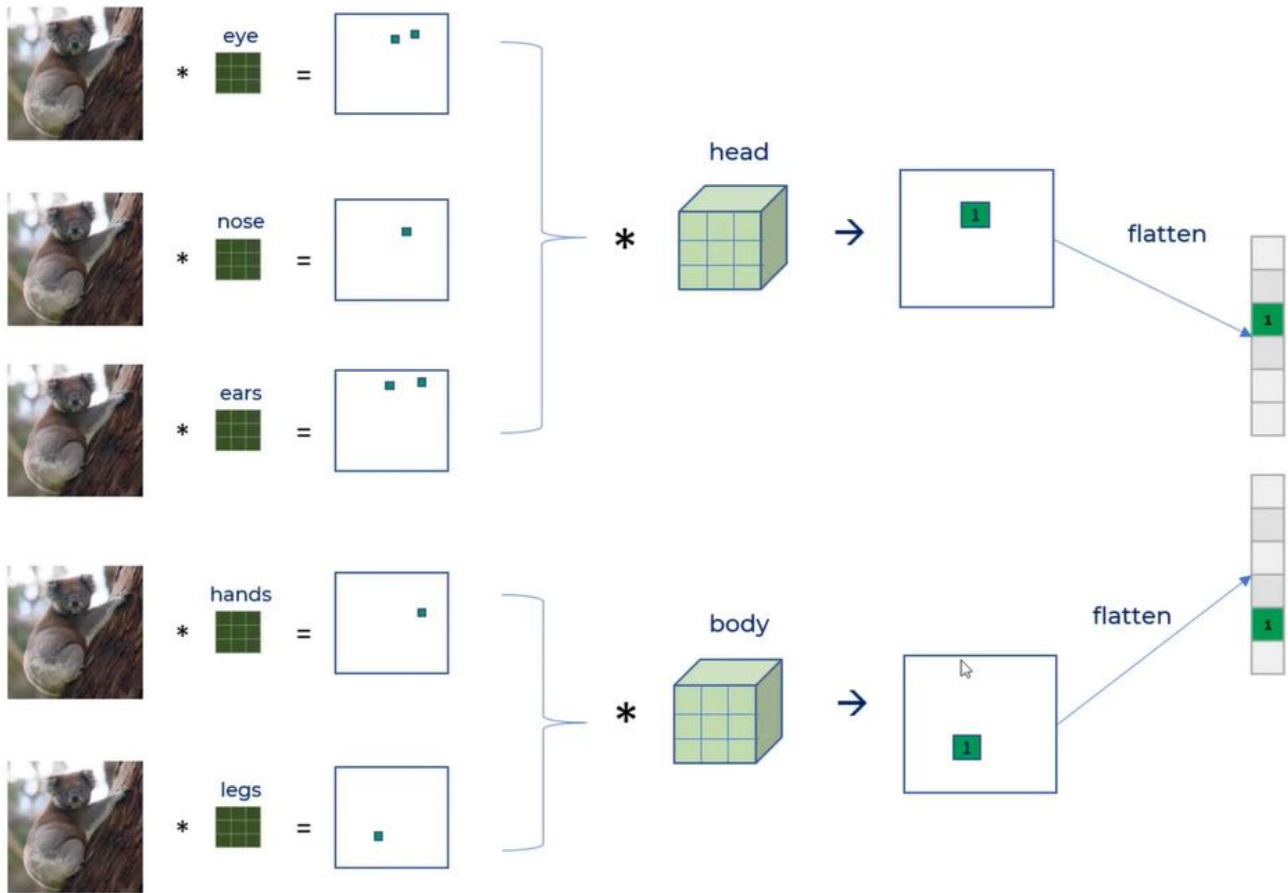
 $=$

	1	

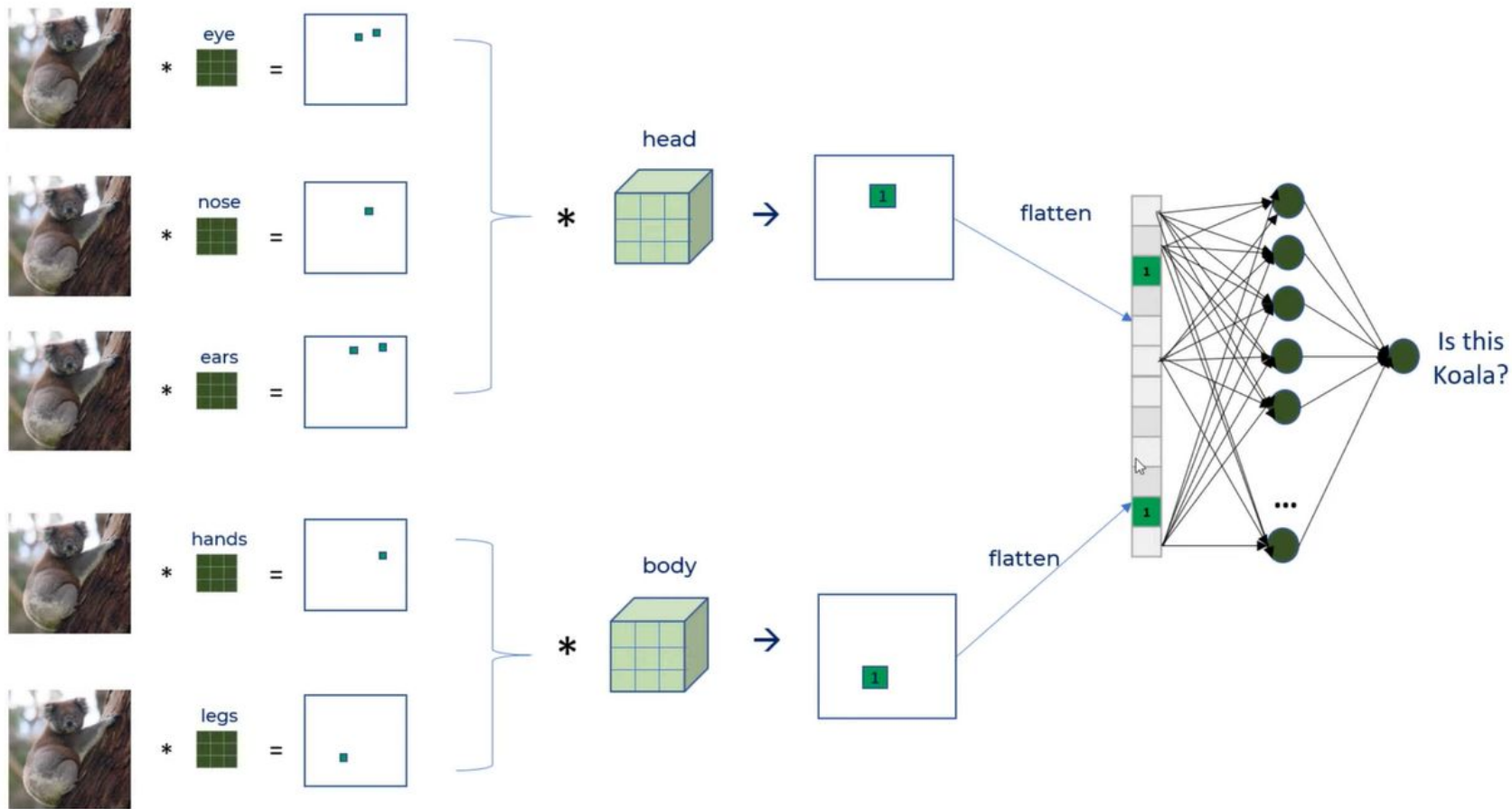
Pattern Detector - for all 3 patterns



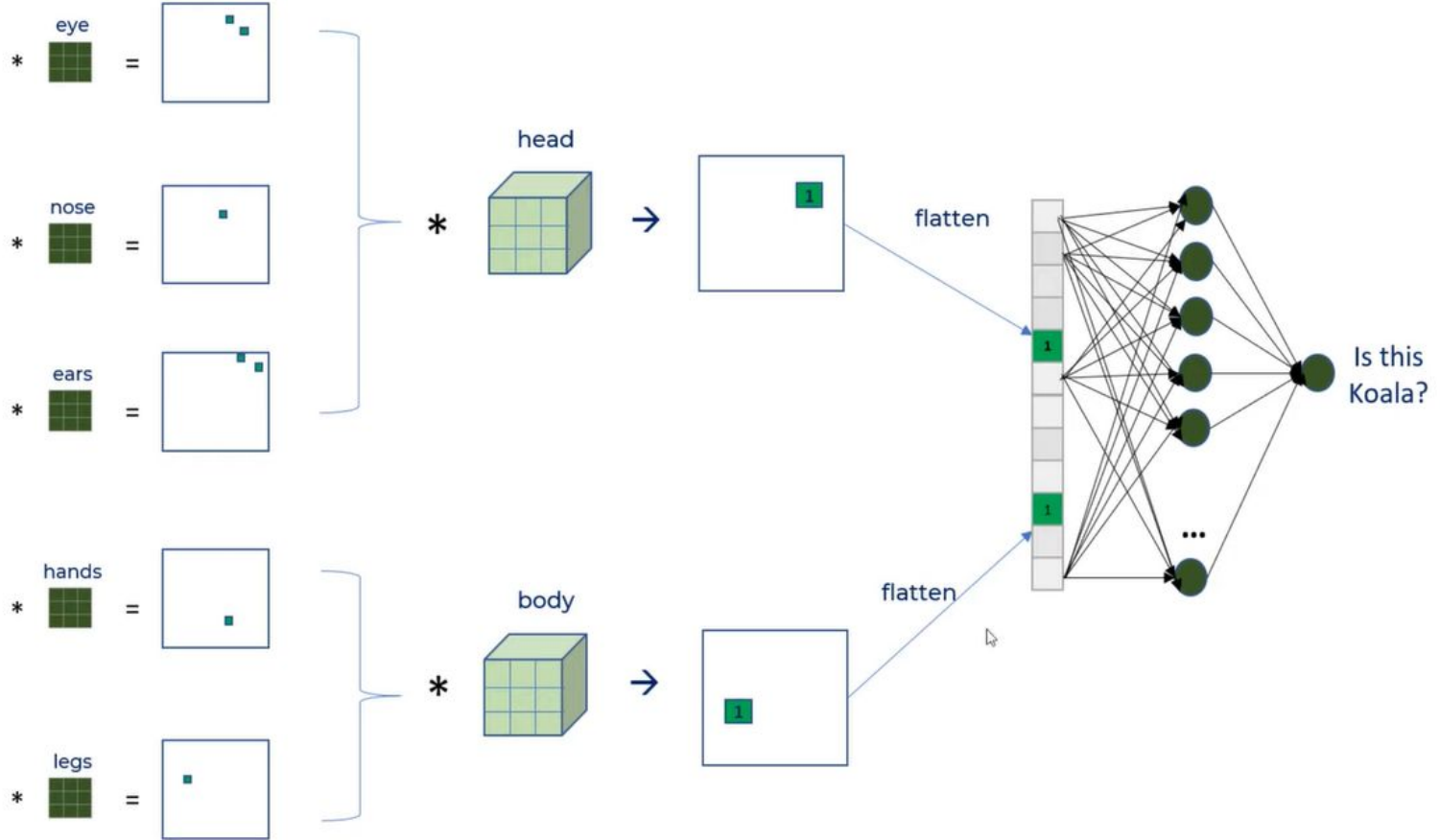




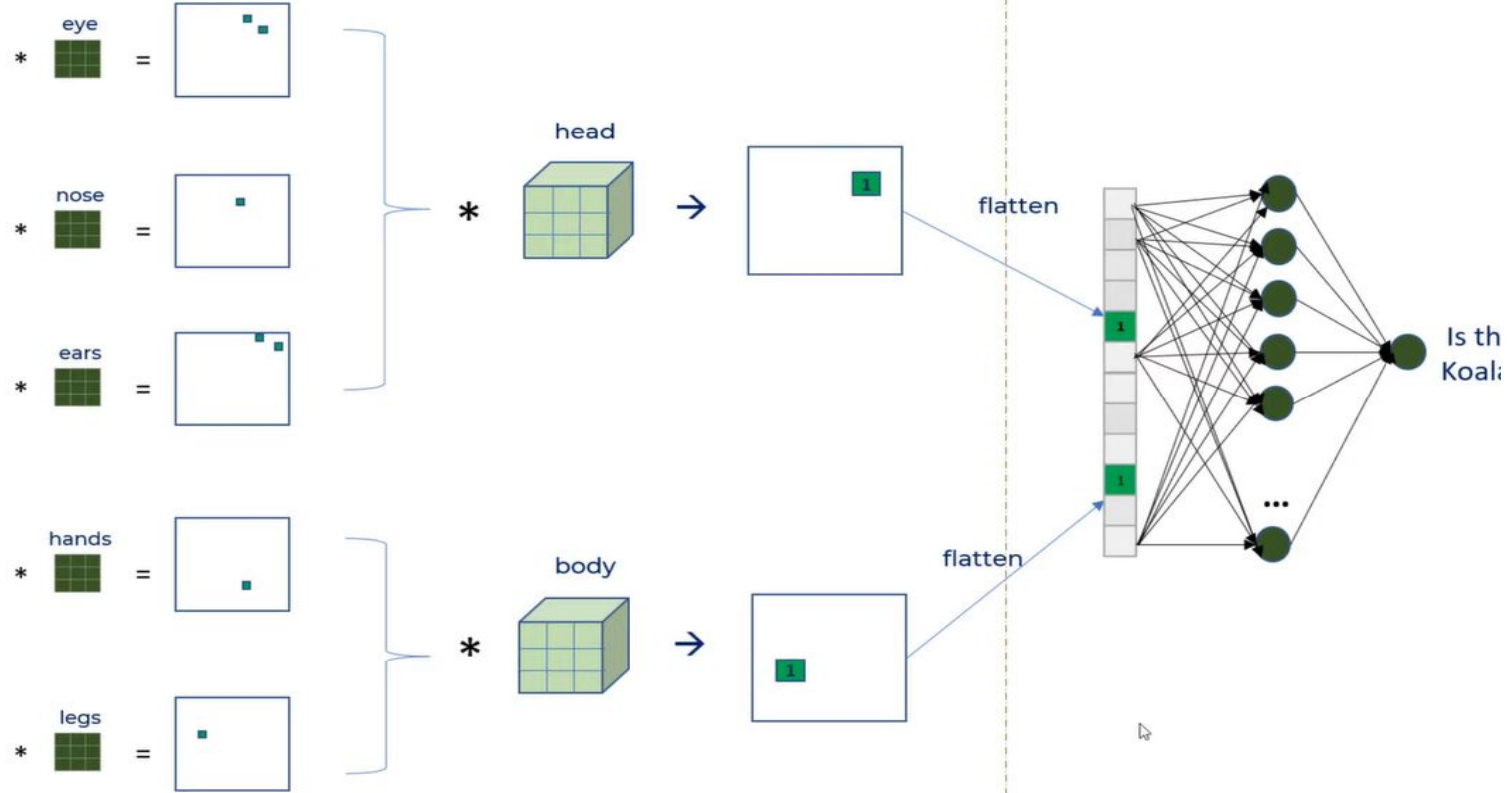
Fully Connected Dense Neural Network



Sleeping Koala - Position is changed



2 Phases in CNN - Feature Extraction + Classification



Feature Extraction

Classification

Activation Layer (ReLU)

-1	1	1	1	-1
-1	1	-1	1	-1
-1	1	1	1	-1
-1	-1	-1	1	-1
-1	-1	-1	1	-1
-1	-1	1	-1	-1
-1	1	-1	-1	-1

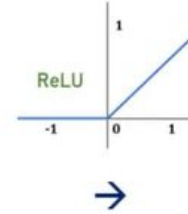
*

Loopy pattern
filter

1	1	1
1	-1	1
1	1	1



-0.11	1	-0.11
-0.55	0.11	-0.33
-0.33	0.33	-0.33
-0.22	-0.11	-0.22
-0.33	-0.33	-0.33



0	1	0
0	0.11	0
0	0.33	0
0	0	0
0	0	0

ReLU : To bring non-linearity in our model

ReLU helps with making
the model nonlinear

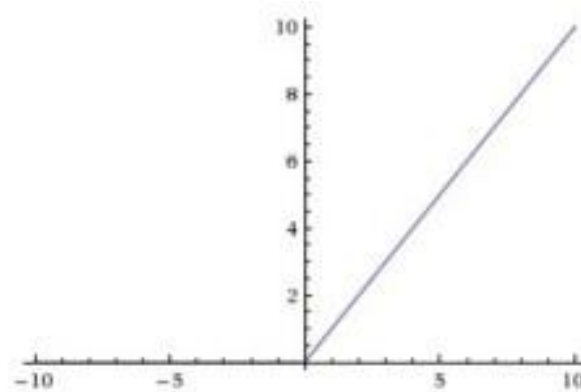


Convolutional Neural Networks (CNNs)

Activation Layer (ReLU)

- An additional operation called Rectified Linear Unit (ReLU) has been used after every Convolution operation

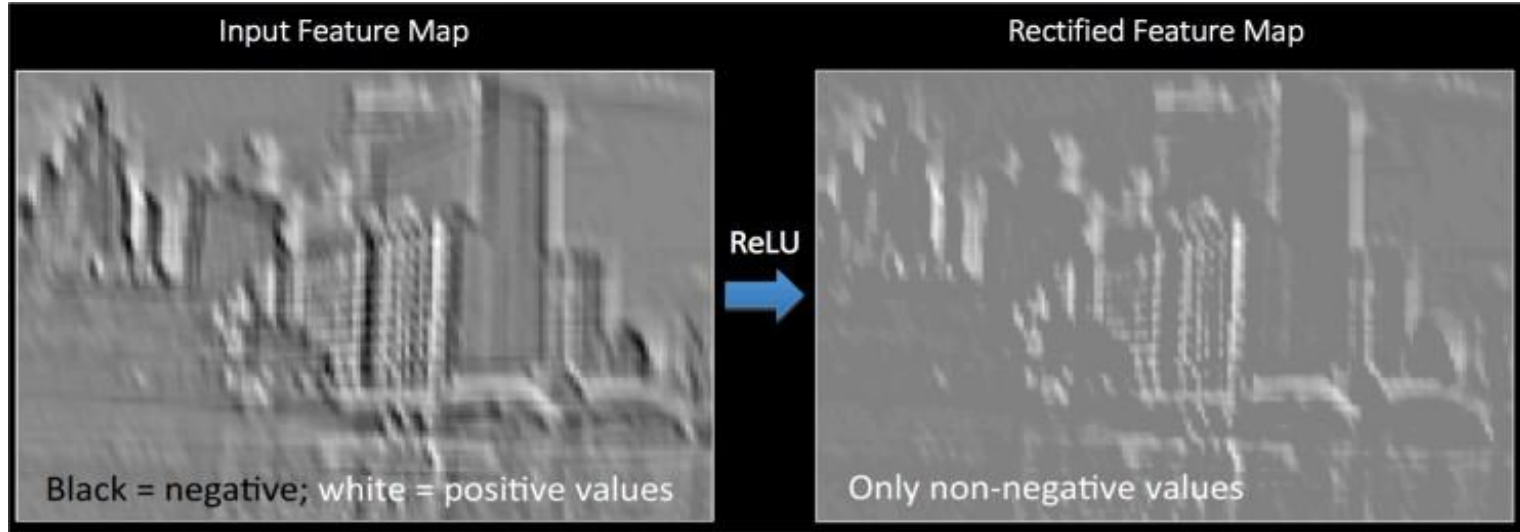
$$\text{Output} = \text{Max}(\text{zero}, \text{Input})$$



- Basically, ReLU is an element wise operation (applied per pixel) and replaces all negative pixel values in the feature map by zero
- The purpose of ReLU is to introduce non-linearity to the network

Convolutional Neural Networks (CNNs)

Activation Layer (ReLU)



Other non-linear functions such as *tanh* or *sigmoid* can also be used instead of ReLU, but **ReLU has been found to perform better in most situations.**

We didn't
address the issue
of too much
computation

This neural network becomes complex when image size is large



Image size = 1920 x 1080 X 3

First layer neurons = 1920 x 1080 X 3 ~ 6 million

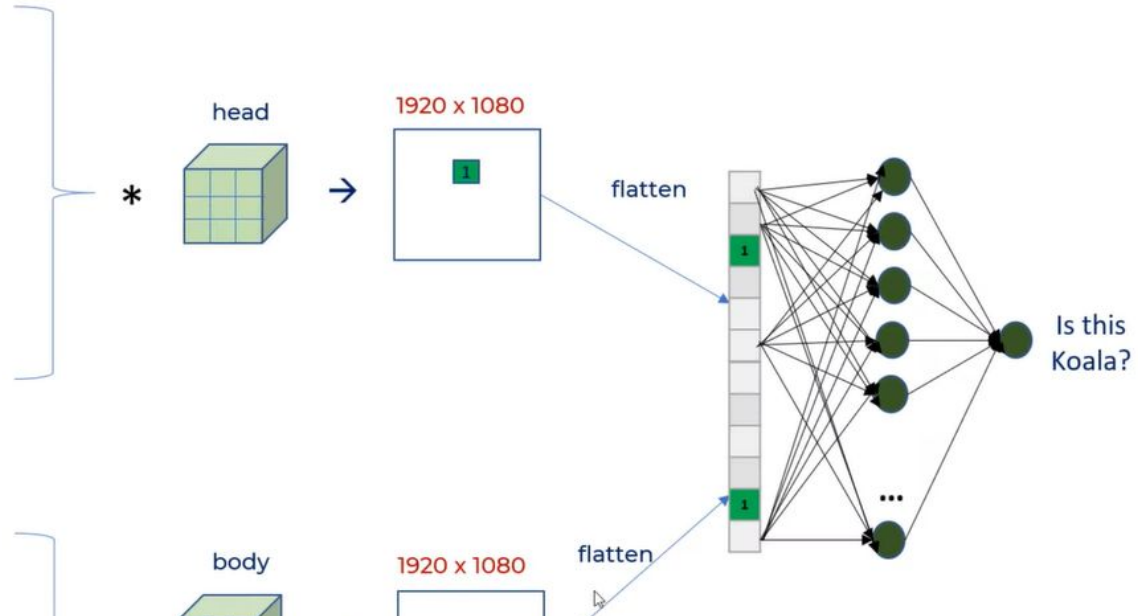
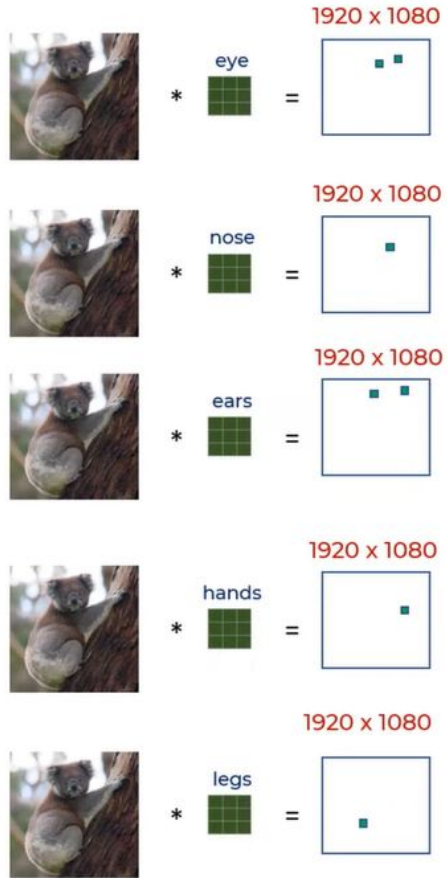
Hidden layer neurons = Let's say you keep it ~ 4 million

Weights between input and hidden layer = 6 mil * 4 mil
= 24 million

24 Million connections in neural network.



Too much computation



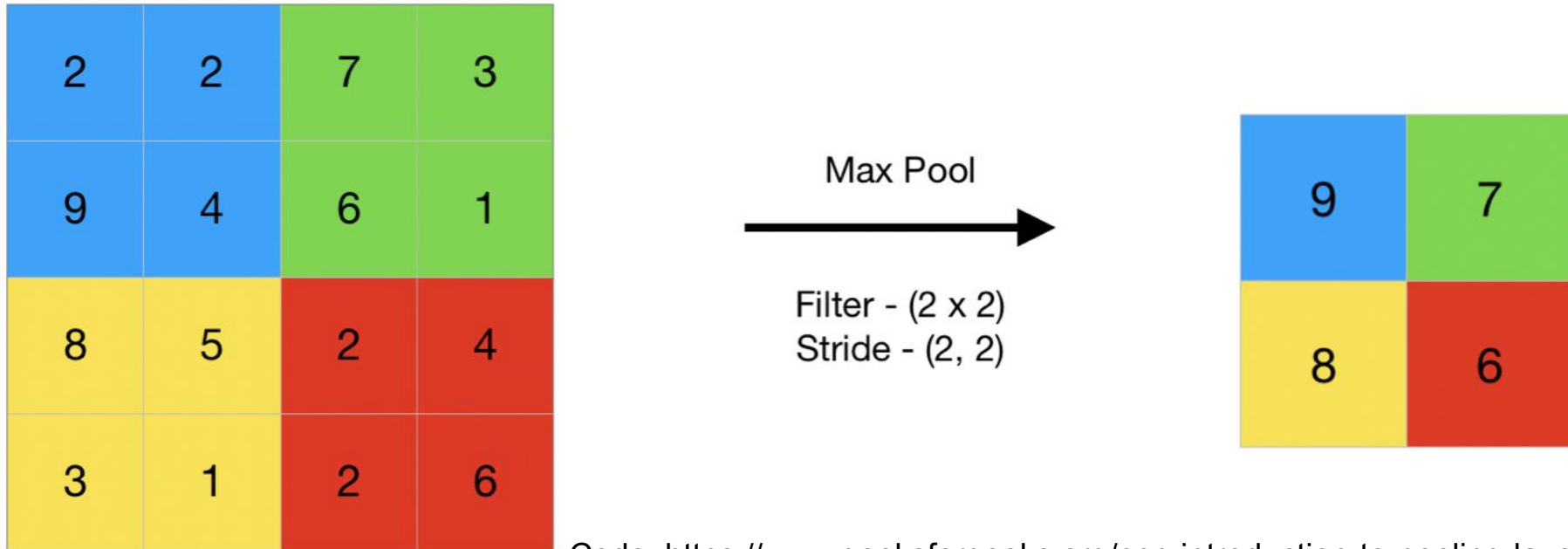
Pooling layer is used to
reduce the size



CNN : Types of Pooling Layers

CNN : Types of Pooling Layers

Max Pooling : It is a pooling operation that selects the maximum element from the region of the feature map covered by the filter. Thus, the output after max-pooling layer would be a feature map containing the most prominent features of the previous feature map.



Code: <https://www.geeksforgeeks.org/cnn-introduction-to-pooling-layer/>

CNN : Types of Pooling Layers

Average Pooling : Average pooling computes the average of the elements present in the region of feature map covered by the filter. Thus, while max pooling gives the most prominent feature in a particular patch of the feature map, average pooling gives the average of features present in a patch.

2	2	7	3
9	4	6	1
8	5	2	4
3	1	2	6

Average Pool



Filter - (2 x 2)
Stride - (2, 2)

4.25	4.25
4.25	3.5

CNN : Types of Pooling Layers

Global Pooling : Global pooling reduces each channel in the feature map to a single value. Thus, an $n_h \times n_w \times n_c$ feature map is reduced to $1 \times 1 \times n_c$ feature map. This is equivalent to using a filter of dimensions $n_h \times n_w$ i.e. the dimensions of the feature map.

Advantage of Pooling layer

1. **Dimensionality reduction:** The main advantage of pooling layers is that they help in reducing the spatial dimensions of the feature maps. This reduces the computational cost and also helps in avoiding overfitting by reducing the number of parameters in the model.
2. **Translation invariance:** Pooling layers are also useful in achieving translation invariance in the feature maps. This means that the **position of an object in the image does not affect the classification result**, as the same features are detected regardless of the position of the object.
3. **Feature selection:** Pooling layers can also **help in selecting the most important features from the input**, as max pooling selects the most salient features and average pooling preserves more information.

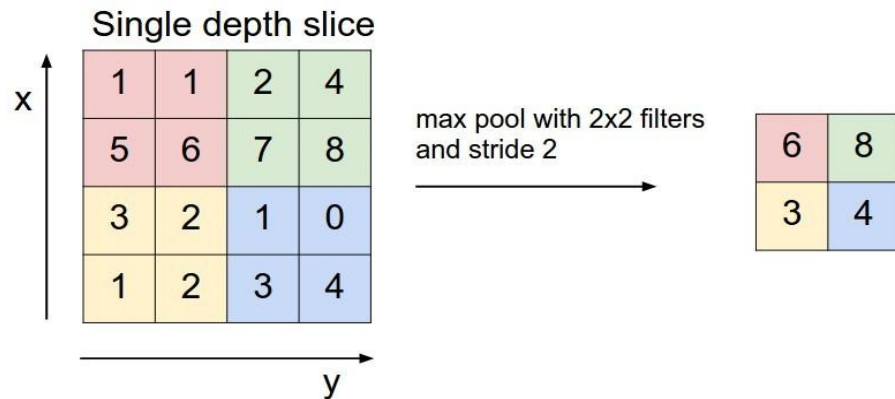
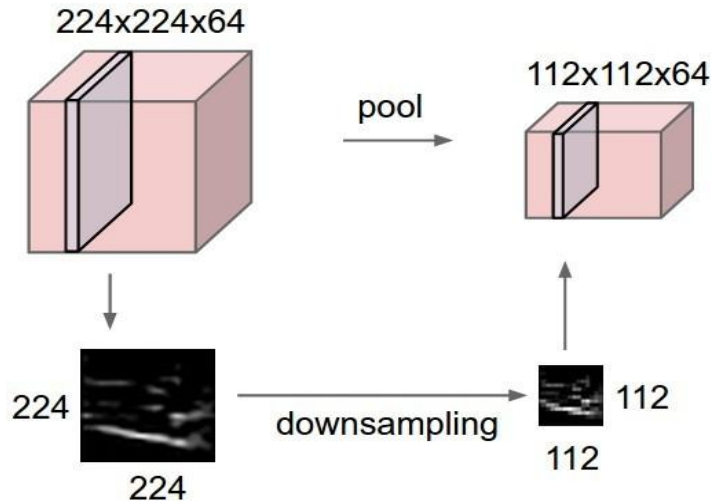
Drawbacks of pooling layer

1. **Information loss:** One of the main disadvantages of pooling layers is that they discard some information from the input feature maps, which can be important for the final classification or regression task.
2. **Over-smoothing:** Pooling layers can also cause over-smoothing of the feature maps, which can result in the loss of some fine-grained details that are important for the final classification or regression task.
3. **Hyperparameter tuning:** Pooling layers also introduce hyperparameters such as the size of the pooling regions and the stride, which need to be tuned in order to achieve optimal performance. This can be time-consuming and requires some expertise in model building.

Convolutional Neural Networks (CNNs)

Pooling Layer

Pooling layer **downsamples** the volume spatially, **independently** in **each depth** slice of the input



The most common downsampling operation is **max**, giving rise to **max pooling**, here shown with a stride of 2

Pooling Layer : Max Pooling + Strides

5	1	3	4
8	2	9	2
1	3	0	1
2	2	2	0

Max Pooling

8	9
3	2

2 by 2 filter with stride = 2

Pooling Layer : Max Pooling + Strides

0	1	0
0	0.11	0
0	0.33	0
0	0	0
0	0	0

1	

2 by 2 filter with stride = 1

Filter + ReLU + Max Pooling

-1	1	1	1	-1
-1	1	-1	1	-1
-1	1	1	1	-1
-1	-1	-1	1	-1
-1	-1	-1	1	-1
-1	-1	1	-1	-1
-1	1	-1	-1	-1

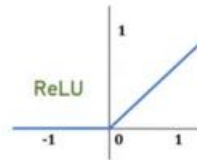
*

Loopy pattern
filter

1	1	1
1	-1	1
1	1	1



-0.11	1	-0.11
-0.55	0.11	-0.33
-0.33	0.33	-0.33
-0.22	-0.11	-0.22
-0.33	-0.33	-0.33



0	1	0
0	0.11	0
0	0.33	0
0	0	0
0	0	0

Max
pooling



For Shifted position of pixels: Filter + ReLU + Max Pooling

Shifted 9 at
different position

1	1	1	-1	-1
1	-1	1	-1	-1
1	1	1	-1	-1
-1	-1	1	-1	-1
-1	-1	1	-1	-1
-1	1	-1	-1	-1
1	-1	-1	-1	-1

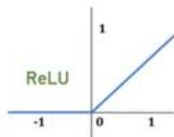
Loopy pattern
filter

*

1	1	1
1	-1	1
1	1	1



1	-0.11	-0.11
0.11	-0.33	0.33
0.33	-0.33	-0.33
-0.11	-0.55	-0.33
-0.55	-0.33	-0.55



1	0	0
0.11	0	0.33
0.33	0	0
0	0	0
0	0	0

Max
pooling



1	0.33
0.33	0.33
0.33	0
0	0

Pooling Layer : Average Pooling + Strides

5	1	3	4
8	2	9	2
1	3	0	1
2	2	2	0

4	4.5
2	0.75

But MaxPoiling is generally used more

Benefits of pooling

Reduces
dimensions &
computation

Reduce overfitting
as there are less
parameters

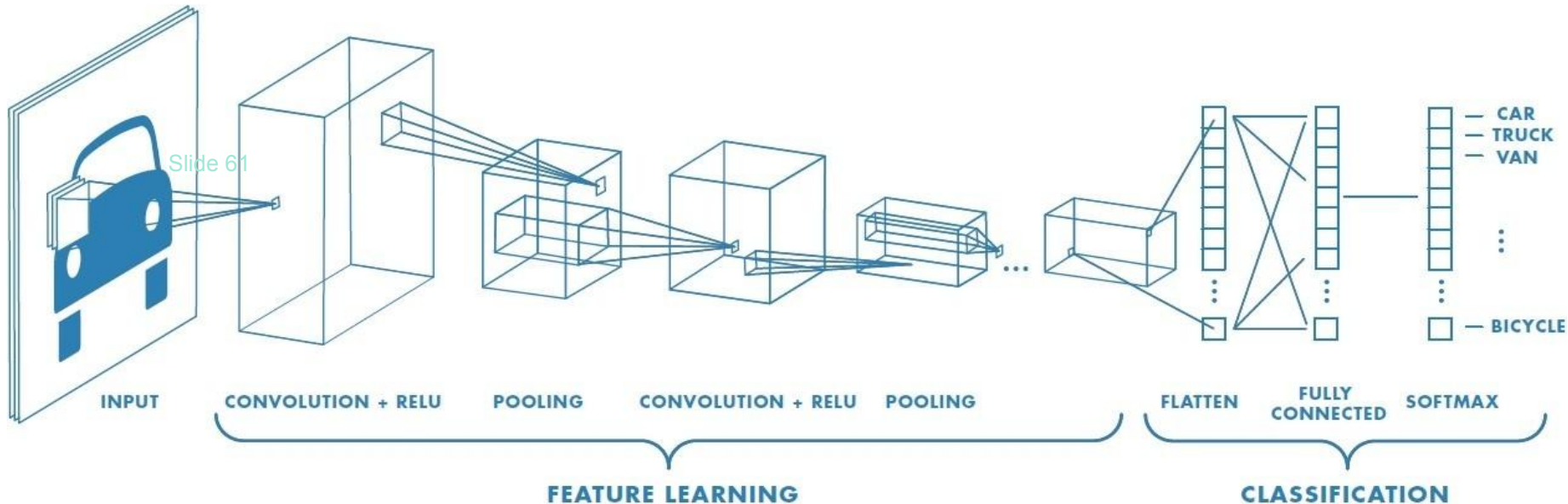
Model is tolerant
towards variations,
distortions

Distortion - Filtering noise

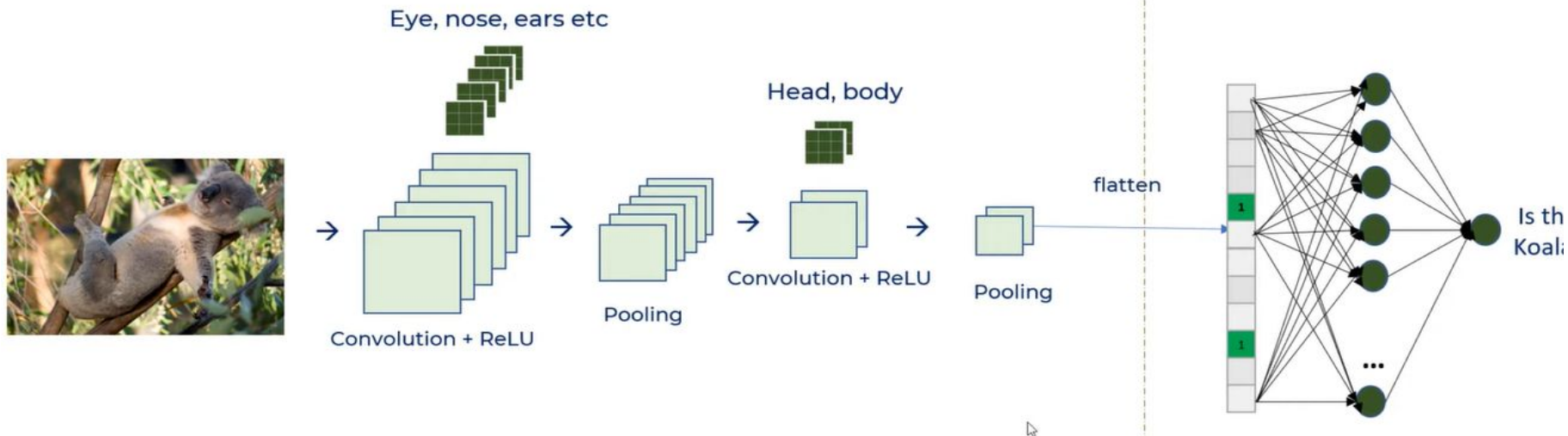
Convolutional Neural Networks (CNNs)

Fully Connected Layer

Neurons in a fully connected layer have full connections to all activations in the previous layer, as seen in regular neural networks



Fully Connected Layer



Main Part

Feature Extraction

Same like ANN

Classification

Convolutional Neural Networks (CNNs)

- Convolutional neural network is one of the main categories to do images recognition, images classifications. Objects detections, recognition faces
- It has 5 layers.
 1. Input layer
 2. Convolutional layer
 3. Pooling layer
 4. Fully connected layer.
 5. Output layer.

Convolution

- Connections sparsity reduces overfitting
 - Conv + Pooling gives location invariant feature detection
 - Parameter sharing
- Connection sparsity - Not every node is connected with other node like in ANN = Dense network
 - Here we have filter to move around the image.
 - We are not affecting whole image and only one local region at a time.

Convolution

- Connections sparsity reduces overfitting
- Conv + Pooling gives location invariant feature detection
- Parameter sharing

ReLU

- Introduces nonlinearity
- Speeds up training, faster to compute

Convolution

- Connections sparsity reduces overfitting
- Conv + Pooling gives location invariant feature detection
- Parameter sharing

ReLU

- Introduces nonlinearity
- Speeds up training, faster to compute

Pooling

- Reduces dimensions and computation
- Reduces overfitting
- Makes the model tolerant towards small distortion and variations

What about - Image Rotation, Thickness ?

Rotation



Thickness



CNN by itself doesn't take care of rotation and scale

- You need to have rotated, scaled samples in training dataset
- If you don't have such samples than use data augmentation methods to generate new rotated/scaled samples from existing training samples

Thank You